NG POLAND 2026 · ANGULAR 22

Re-imagining Module Federation in Angular

Beyond the default toolchain.

Andres Gesteira · Lead Frontend Engineer, VirtaMed Zürich

x.com/GeorgeKaplan_G · linkedin.com/in/andres-gesteira-26a95813

`github.com/kaplan81/re-imaginining-ng-module-deferation`

This is not a TypeScript decorator

```
@Component({
  selector: 'cat-bean-grid',
  imports: [CurrencyPipe, DecimalPipe],
  templateUrl: './bean-grid.component.html',
  styleUrl: './bean-grid.component.scss',
})
export class BeanGridComponent { … }
```

0 - ANGULAR COMPILERS

It is an Angular definition

```
class BeanGridComponent {
  items = input.required(); // your code, unchanged
  stockState(bean) { return bean.stockKg < 60 ? "low" : … } // unchanged

  static ɵfac = (t) => new (t || BeanGridComponent)(); // added
  static ɵcmp = ɵɵdefineComponent({ // added
    type: BeanGridComponent,
    selectors: [["cat-bean-grid"]], inputs: { items: [1, "items"] },
    template: function (rf, ctx) {
      rf & 1 && ( ɵɵdomElementStart(0, "ul", 0), ɵɵrepeaterCreate(1, …),
ɵɵdomElementEnd() );
      rf & 2 && ( ɵɵadvance(), ɵɵrepeater(ctx.items()) );
    }
  });
}
```

rf & 1 — create. Runs once. Builds the DOM.

rf & 2 — update. Runs on every change detection.

0 - ANGULAR COMPILERS

ngc is a superset of tsc

`NgtscProgram`	wraps	`ts.Program`
`NgCompilerHost`	wraps	`ts.CompilerHost`
`NgCompiler`	drives	every Angular-specific phase

The Angular compiler **wraps the TypeScript compiler** — so everything Angular does to your code has to fit through TypeScript's own compiler API.

ngc compiles the decorator

#	Phase	What it does	Half
1	Detect	Find decorated classes	frontend
2	Analyze	Read decorators and their metadata	frontend
3	Resolve	Link each reference to the class it points at, across all files	frontend
4	Type-check components only	Templates become Type Check Blocks: TypeScript that tsc checks, then drops	backend
5	Compile	Build definition functions. ɵɵdefineComponent() will be assigned to ɵcmp nothing is written yet	backend
6	Emit	Put definitions in the class, delete the decorator, print the output runs as a ts.Transformer	backend

All six run for every decorated class — component, directive, pipe, service.

The backend produces **two outputs from the same analysis**: the type-check blocks, and the code that replaces the decorator.

Frontend — understands decorators and how they are interconnected. Global and slow.

Backend — transforms file by file. Local and fast.

0 - ANGULAR COMPILERS

The APIs Angular needs are gone

> “The Angular compiler implements its own code transformations via the ts.Transformer API **which is not available cross-language**.”
>
> Angular blog · *An update on Angular’s TypeScript 7-powered Compiler* · September 2026

1. TypeScript 7’s compiler is a program written in **Go**. Stable since July 2026.
2. ts.Transformer is only possible while both sides are JavaScript.
3. Go and JavaScript **cannot share objects**. So those APIs cannot be made public again

0 - ANGULAR COMPILERS

Angular Preprocessor works out of the bundler



Prototype. In `angular/main` since 8 October 2026 — not released as of 24 October 2026.

1 - MODULES

Let us start from the beginning

```
import { Component } from '@angular/core'
```

That quoted string is a **specifier**. It names a module — in practice, a file — but says nothing about where that file is.

Something has to turn it into a file a browser can fetch.

The four types of specifier

Type	Example
Relative URL path	`./util.js`
Absolute URL path	`/lib/util.js`
Full URL	`https://cdn.example/x.js`
Bare specifier	`@angular/core`

The first three tell a browser where the module's file is. **A bare specifier does not**.

1 - MODULES

Static imports and dynamic imports

STATIC

```
import { Component } from '@angular/core';
```

Only at the top of a file.
The specifier **must be a literal string**, so the whole graph is known before anything runs.
This is what tree-shaking is built on.

DYNAMIC

```
const routes = await import('./catalog.routes');
```

Anywhere code can go. Returns a promise.
Resolved when the line runs.
The specifier **can be a variable**.

A static specifier is known **before the program starts**. A dynamic one does not have to be.

1 - MODULES

Three module resolution times

When	Who resolves it
Build time	the bundler
Load time	the browser’s loader
Runtime	a loader you ship

1 - MODULES

Build time — the bundler resolves it

you wrote	`'@angular/core'`
it shipped	`_angular_core.AnS7MGp98i.js`

The bundler had `node_modules` on a disk. **The browser never will.**

Load time — the browser resolves it

```
<script type="importmap">
{ "imports": {
  "@angular/core": "/assets/core-DG2k.js",
  "rxjs": "/assets/rxjs-9Fz1.js"
} }
</script>
```

No library. This is the browser's own loader — and the map has to be in the page **before the first import runs**.

A web standard, available in every major browser since 2023.

Runtime — a loader you ship resolves it

```
const url = registry.lookup('catalog');
const mod = await import(url);
```

That URL **did not exist when the application was built**. It was fetched, or computed, while the app was already running.

Only possible with a dynamic import — a static one needs a literal.

Modules can be federated

CODE SPLITTING

Chunks from your own build.
Fetched when they are needed.

FEDERATION

Chunks from a different build,
on a different server.

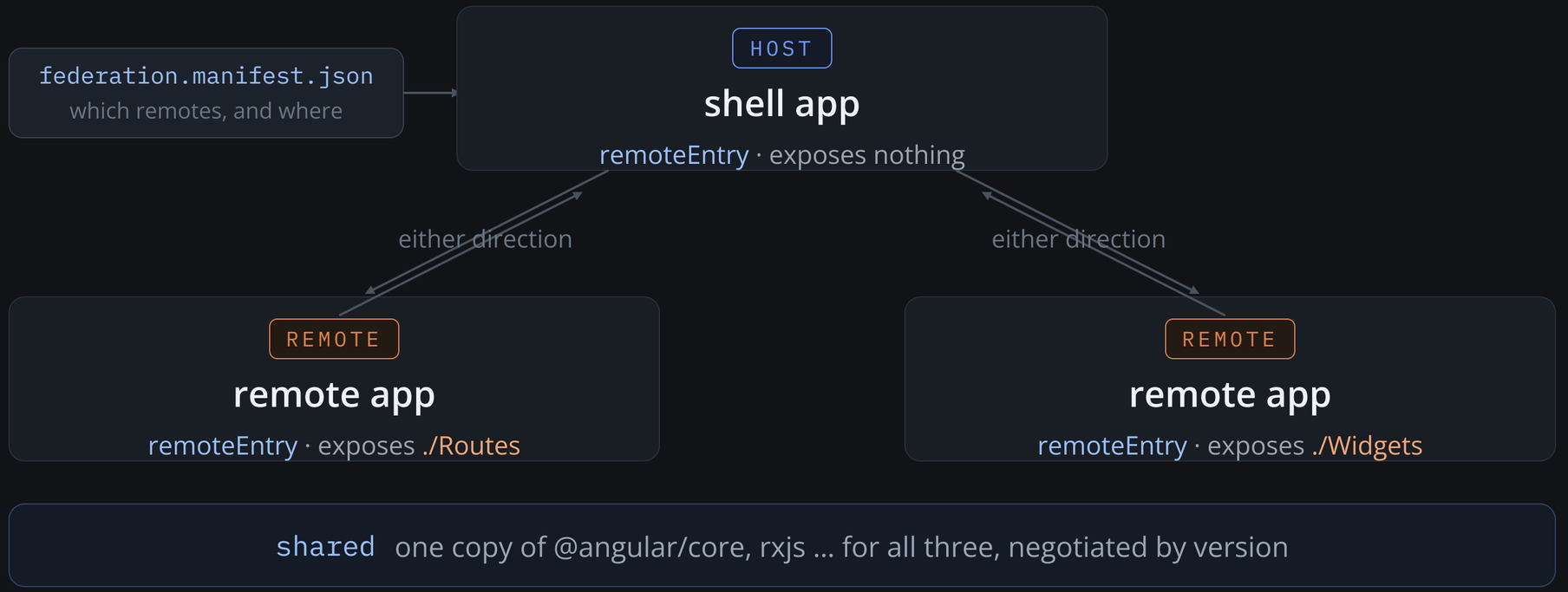
Code splitting: **one build**. Federation: **several builds, interconnected**.

Each federated module has its own build, its own deployment, its own URL.

The one that loads is the **host**. The one being loaded is the **remote**.

2 - MODULE FEDERATION

Module Federation is a mental model



Module Federation vs Native Federation

	Module Federation @module-federation/*	Native Federation @angular-architects/native-federation
Who resolves	its own **loader**	the browser's **loader**
Module paths are	consulted every time	written once, at boot
Override a remote	registerRemotes(..., {force:true})	not possible — the first mapping wins

A build tool is not a bundler

BUILD TOOL — **Contains a bundler**, plus a dev server, TypeScript, stylesheets, assets.

BUNDLER — Walks the module graph and splits it into **chunk files.**

The build tool you name	The bundler doing the work
Angular CLI	esbuild
Vite	Rolldown
Rsbuild	Rspack

A build tool has **defaults**. For example Rsbuild compiles .ts with no configuration whereas Rspack needs a rule pointing at a loader.

Three build pipelines

Pipeline	The Angular plugin it loads	Bundler	Federation
angular.json ng build · the default	@angular/build	esbuild	Native Federation
rspack.config.ts rspack build	@nx/angular-rspack	Rspack	Module Federation
vite.config.ts vite build	@analogjs/vite-plugin-angular	Rolldown	Module Federation

A custom pipeline is **a config file and a plugin**. Same five applications, same feature code, three times.

Vite pipeline

Why it is probably not an option

There is no federation reason to use Vite, unless you are very interested in **its ecosystem**.

Chunk URL rewriting is not possible.

Unused shared modules are **not automatically pruned**.

There is no real advantage over the **Angular CLI** capabilities.

Rspack pipeline

Why it could be an option

Runtime plugins for extending federation behaviour.

Hooks around the **federation lifecycle**.

Chunk URL rewriting is possible, unlike in Native Federation.

The loader itself, remoteEntry.js, is **replaceable**.

Seamless integration with **Zephyr Cloud**.

3 - BUNDLERS

What ngp changes about all of this

It compiles **one file at a time**, so a bundler plugin can compile only the files a page actually reaches. That is **lazy compilation**, and it takes over part of what **Module Federation** is for.

It can do **separate builds**, like Module Federation does — but not **separate deployments**, because the minimal deployable unit is **the app**.

TODAY	Every bundler that is not esbuild needs an adapter to host the Angular compiler. The adapter (or plugin) sets the ceiling.
WITH NGP	The compiler leaves a file. The bundler reads TypeScript. **There is nothing left to adapt.**

It makes Angular more pluggable.

Demo time

Five applications, three pipelines, fifteen dev servers.